

# Operating Systems and System Programming

## Skill-7

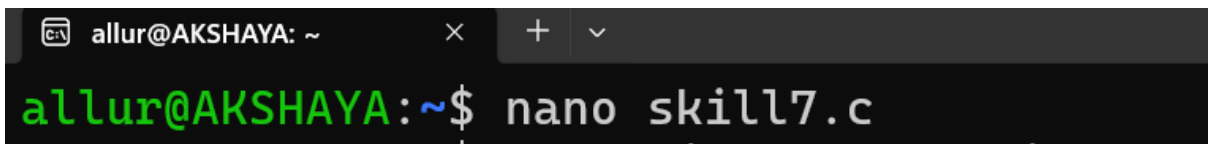
### Aim

To implement process synchronization and child process monitoring using `waitpid()`, retrieve and parse the `PATH` environment variable, locate executable commands, verify permissions, and handle missing commands.

### Algorithm

1. Start the program.
2. Create a child process using `fork()`.
3. In the child process, read the `PATH` environment variable.
4. Split `PATH` into individual directories.
5. Search each directory for the given command.
6. Check whether the command exists and has execution permission.
7. Display the executable path or an error message.
8. In the parent process, use `waitpid()` to wait for the child.
9. Display the child's exit status.
10. Stop the program.

### Step-1:

A terminal window with a dark background. The title bar shows 'allur@AKSHAYA: ~' and window control buttons. The prompt is 'allur@AKSHAYA:~\$' in green. The command 'nano skill7.c' is being typed in white.

### Step-2:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill7.c *  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid;  
    int status;  
    char command[100];  
    char path[4096];  
    char fullpath[1200];  
    char *dir;  
    char *env;  
  
    printf("Enter command: ");  
    scanf("%99s", command);  
  
    pid = fork();  
  
    if (pid < 0)  
    {  
        perror("Fork failed");  
        return 1;  
    }  
  
    if (pid == 0)  
    {  
        env = getenv("PATH");  
  
        if (env == NULL)  
        {  
            printf("PATH not found\n");  
            exit(1);  
        }  
  
        snprintf(path, sizeof(path), "%s", env);  
        dir = strtok(path, ":");  
  
        while (dir != NULL)  
        {  
            snprintf(fullpath, sizeof(fullpath),  
                "%s/%s", dir, command);  
            if (access(fullpath, X_OK) == 0)  
            {  
                printf("Executing: %s\n", fullpath);  
                execve(fullpath, &command, env);  
            }  
            dir = strtok(NULL, ":");  
        }  
        printf("Command not found\n");  
        exit(1);  
    }  
    wait(&status);  
    printf("Process exited with status: %d\n", status);  
}
```

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill7.c *  
  
if (pid < 0)  
{  
    perror("Fork failed");  
    return 1;  
}  
  
if (pid == 0)  
{  
    env = getenv("PATH");  
  
    if (env == NULL)  
    {  
        printf("PATH not found\n");  
        exit(1);  
    }  
  
    snprintf(path, sizeof(path), "%s", env);  
    dir = strtok(path, ":");  
  
    while (dir != NULL)  
    {  
        snprintf(fullpath, sizeof(fullpath),  
            "%s/%s", dir, command);  
        if (access(fullpath, X_OK) == 0)  
        {  
            printf("Executing: %s\n", fullpath);  
            execve(fullpath, &command, env);  
        }  
        dir = strtok(NULL, ":");  
    }  
    printf("Command not found\n");  
    exit(1);  
}
```

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill7.c *  
    snprintf(fullpath, sizeof(fullpath),  
             "%s/%s", dir, command);  
  
    if (access(fullpath, X_OK) == 0)  
    {  
        printf("Command found: %s\n", fullpath);  
        exit(0);  
    }  
  
    dir = strtok(NULL, ":");  
}  
  
printf("Command not found: %s\n", command);  
exit(1);  
}  
else  
{  
    printf("Parent waiting for child...\n");  
    waitpid(pid, &status, 0);  
  
    if (WIFEXITED(status))  
        printf("Child exit status: %d\n",  
               WEXITSTATUS(status));  
}
```

```
    if (WIFEXITED(status))  
        printf("Child exit status: %d\n",  
               WEXITSTATUS(status));  
}  
  
return 0;  
}
```

### Step-3:

```
allur@AKSHAYA:~$ gcc skill7.c -o skill7  
allur@AKSHAYA:~$ ./skill7  
Enter command: ls  
Parent waiting for child...  
Command found: /usr/bin/ls  
Child exit status: 0  
allur@AKSHAYA:~$ ./skill7  
Enter command: xyz  
Parent waiting for child...  
Command not found: xyz  
Child exit status: 1  
allur@AKSHAYA:~$
```

## **Result**

Process synchronization and child process monitoring were successfully implemented using `fork()` and `waitpid()`. The `PATH` environment variable was retrieved and parsed to locate executable commands, verify execution permissions, and handle missing commands.